

Autonomous Agent Suite: Architecture & Best Practices

This report addresses six design questions for a suite of autonomous agents running on macOS launchd schedules, reading and writing to a shared Obsidian vault via the Claude Agent SDK.

1. Multi-Agent Filesystem Coordination

Your architecture already has a strong foundation here. The `vault_inject` MCP tool performs anchor-based PATCH operations — appending content below HTML comment anchors without replacing existing content. This means multiple agents and interactive sessions can write to the same daily note without stepping on each other, as long as they target different anchors.[1]

Why Race Conditions Are Low-Risk (But Not Zero)

Your agents run on staggered launchd schedules — 6am, 8am, 5pm, 7pm, etc. Since launchd fires agents sequentially within your user session and each agent targets distinct anchors (e.g., `<!-- priorities -->` vs `<!-- reflection -->` vs `<!-- health -->`), the probability of two agents writing to the same file region simultaneously is very low. The risk surfaces in two scenarios: (1) an agent run takes longer than expected and overlaps with the next scheduled agent, or (2) you manually trigger an agent while a scheduled one is running.

Advisory File Locking with `filelock`

For the cases where collision *could* happen, the `filelock` Python library provides clean, cross-platform inter-process locking. The pattern is straightforward:[2][3]

```
from filelock import FileLock, Timeout
```

```
def safe_vault_write(file_path, anchor, content):
    lock = FileLock(f"{file_path}.lock", timeout=30)
    try:
        with lock:
            inject_at_anchor(file_path, anchor, content)
    except Timeout:
        logger.error(f"Could not acquire lock on {file_path} after 30s")
        # Record failure in agent-run-history.csv for visibility
```

Key implementation details:

- Create a .lock file alongside the vault file (e.g., 2026-02-22.md.lock), never lock the actual file you're writing to.[3]
- Use a timeout (30 seconds is reasonable) so agents don't hang indefinitely.
- On timeout failure, log to agent-run-history.csv and exit gracefully — the next scheduled run will pick up the work.
- File locks on Unix are advisory, meaning all cooperating processes must agree to use them. Since you control all agent code, this is fine.[4][5]

Per-Anchor Locking (Optional Optimization)

If you want finer-grained concurrency (e.g., two agents writing different anchors in the same file simultaneously), you can lock at the anchor level rather than the file level:

```
lock = FileLock(f"{file_path}.{anchor}.lock", timeout=30)
```

This is overkill for your current schedule staggering but becomes useful if you add more agents targeting the same daily note.

Practical Recommendations

- Add filelock to your pyproject.toml dependencies.
- Wrap vault_io.inject_at_anchor() with locking logic in lib/vault_io.py so every agent automatically gets safe writes.
- Add a --force flag for manual runs that skips the lock (useful for debugging, dangerous in production).

- Git-track your vault — even if a race condition corrupts a file, you can recover from the last commit. The `jj` commit pattern from Claude Code Stop hooks is instructive here.[6]

2. Preserve Session Agent: Capturing Session Context

This is one of your most architecturally interesting agents. The Stop hook fires when a Claude Code session ends, and the data available at that point is rich enough to build a robust context-capture pipeline.

What Data Is Available at Session End

When a Stop hook fires, Claude Code passes a JSON payload via `stdin` containing:[7][8][9]

Field	Description
<code>session_id</code>	Unique identifier for the session
<code>transcript_path</code>	Absolute path to the JSONL conversation file
<code>cwd</code>	Working directory when the hook fired
<code>permission_mode</code>	Current permission mode
<code>hook_event_name</code>	Always "Stop" for this hook

The `transcript_path` points to a JSONL file stored in `~/.claude/projects/` where each line is a JSON object representing a conversation turn.

The structure includes:[10][11]

- **User messages** with the full prompt text
- **Assistant messages** with Claude's complete responses
- **Tool use records** showing every file read, write, edit, bash command, etc.
- **Tool results** with the output of each tool call
- Timestamps and UUIDs for each turn[11]

Recommended Architecture: Two-Phase Capture

Phase 1 — Stop Hook (fast, synchronous): The Stop hook should be lightweight because it runs synchronously before the CLI exits. Use it to copy/snapshot the transcript path and basic metadata, then hand off to the agent:

```
#!/usr/bin/env python3
"""Stop hook: queue session for Preserve Session agent."""
import json, sys, shutil
from pathlib import Path

hook_input = json.load(sys.stdin)
transcript = Path(hook_input["transcript_path"])
queue_dir = Path.home() / "Code-Brain/claude-code-superuser-pack/agents-sc
queue_dir.mkdir(exist_ok=True)

# Write a job file the agent will pick up
job = {
    "session_id": hook_input["session_id"],
    "transcript_path": str(transcript),
    "cwd": hook_input["cwd"],
    "timestamp": datetime.now().isoformat()
}
(queue_dir / f"{hook_input['session_id']}.json").write_text(json.dumps(job))
```

Phase 2 — Agent run (async, via SDK): The Preserve Session agent runs after the hook, reads the transcript JSONL, and sends it to Claude via the SDK's `query()` with a skill prompt that extracts decisions, context, and carry-forward items. This is how Claude-Mem's architecture works — the Stop hook generates summaries using the Agent SDK.[12]

Should the Hook Pass Summary Data or the Agent Read Transcripts?

The agent should read the raw JSONL transcript. Here's why:

- **Transcripts contain everything** — exact code snippets, error messages, tool outputs, and decision rationale that a pre-computed summary might lose.[13]
- **The skill prompt controls extraction** — your `preserve-session SKILL.md` can instruct Claude to extract specific categories (decisions made, files changed, open questions, things to remember) rather than generic summaries.
- **You avoid duplicating summarization logic** between the hook and the agent.

Known Pitfall: The `/clear` Bug

If the user runs `/clear` during a session, the transcript path received by the Stop hook may point to a file that doesn't exist. Your agent should handle this gracefully:[14]

```
if not Path(job["transcript_path"]).exists():
    logger.warning(f"Transcript missing for {job['session_id']} (likely /clear used)")
    record_run(log_dir, "preserve-session", "capture", "skipped", 0, 0, 0,
               "Transcript file missing")
return
```

Parsing JSONL Transcripts

Each line in the JSONL is a message object with role, content, timestamp, and optional tool use/result arrays. You can parse them into structured dataclasses:[11]

```
import json
from pathlib import Path

def parse_transcript(path: Path) -> list[dict]:
    messages = []
    with open(path) as f:
```

```
for line in f:
    if line.strip():
        messages.append(json.loads(line))
return messages
```

For cost efficiency, consider truncating very long transcripts before sending to Claude — the memsearch transcript approach of selecting the most recent N turns or filtering by tool use type is a good model. [13]

3. Financial Data Privacy for Bank CSV Parsing

Your Spending Analysis agent parses bank CSVs to identify recurring charges and track budget. Since this data contains account numbers, merchant names linked to real purchases, and transaction amounts, you need a privacy-conscious pipeline.

Recommended Pattern: Pre-Process, Then Prompt

The safest approach is a **two-stage pipeline** where a Python pre-processing step strips sensitive identifiers before anything reaches the Claude API:

Stage 1 — Local Python pre-processing (no API call):

```
import csv
from pathlib import Path

def sanitize_transactions(csv_path: Path) -> list[dict]:
    """Strip account numbers, normalize merchant names."""
    sanitized = []
    for row in csv.DictReader(open(csv_path)):
        sanitized.append({
            "date": row["Date"],
            "merchant": categorize_merchant(row["Description"]),
            "amount": float(row["Amount"]),
            "category": row.get("Category", "Uncategorized"),
```

```
# OMIT: account_number, card_last_four, balance
})
return sanitized
```

Stage 2 — Claude analyzes the sanitized summary:

The agent passes only categories, amounts, and dates to Claude. The skill prompt asks Claude to identify patterns (recurring charges, spending spikes, subscription candidates) from the pre-processed data.

Why This Architecture Is Better Than Alternatives

- **Anonymization before the API call** means even if Claude's API were compromised or logs were exposed, your raw financial data never left your machine.[15][16]
- **Tokenization** (replacing real account numbers with pseudonyms) preserves analytical utility while removing PII — this is the pattern recommended for PCI DSS compliance.[17]
- **The pre-processor is deterministic Python** — no LLM involved, so it's fast, free, and auditable.
- Claude doesn't need your account number to tell you that you're spending \$247/month on subscriptions. It only needs the category and amount.

Additional Safeguards

- Store raw CSVs in life-systems/finance/ (per your config.toml), which is outside the vault and should be .gitignored.[1]
- The block-secrets.py hook already prevents agents from reading .env files — extend it to also block access to raw CSV paths during the Claude API call.[1]
- Write the agent's output (analysis, budget summaries) to vault/50_sources/finance/ as your config already specifies.[1]
- Consider whether you even need Claude for the categorization step — a local Python dictionary mapping merchant strings to categories might handle 80% of cases, with Claude only processing the ambiguous remainder.

Merchant Categorization Approach

```
KNOWN_MERCHANTS = {
    "SPOTIFY": "Entertainment/Music",
    "NETFLIX": "Entertainment/Streaming",
    "CHATGPT": "AI Tools",
    "ANTHROPIC": "AI Tools",
    "CURSOR": "AI Tools",
    # ... build this over time
}

def categorize_merchant(description: str) -> str:
    desc_upper = description.upper()
    for keyword, category in KNOWN_MERCHANTS.items():
        if keyword in desc_upper:
            return category
    return description # Pass through for Claude to categorize
```

4. launchd Patterns for Network/API Resilience

How launchd Handles Sleep and Missed Schedules

This is one of launchd's strengths over cron. With `StartCalendarInterval`, if your Mac is asleep during the scheduled time, launchd will start the job the next time the computer wakes up. If multiple intervals pass while the system is asleep, they are **coalesced into a single event** on wake. This means your 6am morning agent will still fire when you open your laptop at 6:45am, but you won't get seven queued runs if you were away for a week. [18]

`StartInterval` (interval-based scheduling) behaves differently — missed intervals are simply lost. Since all your agents use calendar-based scheduling, you're on the right track.[18]

Handling API Downtime

launched doesn't know whether your agent succeeded or failed — it just runs the Python script. Your Python wrapper needs retry logic:

```
import time
import subprocess

MAX_RETRIES = 3
RETRY_DELAY = 300 # 5 minutes

async def run_with_retry(mode: str, config: Config):
    for attempt in range(1, MAX_RETRIES + 1):
        try:
            result = await query(prompt, options)
            record_run(config.log_dir, "daily-driver", mode, "success", ...)
            return result
        except (ConnectionError, TimeoutError, subprocess.CalledProcessError):
            logger.warning(f"Attempt {attempt}/{MAX_RETRIES} failed: {e}")
            if attempt < MAX_RETRIES:
                time.sleep(RETRY_DELAY)
            else:
                record_run(config.log_dir, "daily-driver", mode, "failed",
                           0, 0, 0, f"API unavailable after {MAX_RETRIES} attempts")
```

Network Availability Check

Add a pre-flight check before spending time on prompt construction:

```
import urllib.request

def api_available(timeout=10) -> bool:
    try:
        urllib.request.urlopen("https://api.anthropic.com/v1/messages", timeout)
        return True
    except Exception:
        return False
```

launchd Wake-from-Sleep Pattern

For agents that *must* run at specific times (e.g., the morning Daily Driver before your workday starts), use `pmset` to wake the machine:

```
sudo pmset repeat wake MTWRFSU 05:55:00
```

This wakes the Mac at 5:55am, giving `launchd` time to fire the 6:00am agent. Note that Apple removed the GUI for this in macOS Ventura and above, so Terminal is the only way.[19]

Stale Run Detection

Add a helper that checks if the agent already ran today (useful when coalesced wake events fire):

```
def already_ran_today(log_dir: Path, agent: str, mode: str) -> bool:
    history = log_dir / "agent-run-history.csv"
    if not history.exists():
        return False
    today = date.today().isoformat()
    with open(history) as f:
        return any(
            row.startswith(f"{today},") and agent in row and mode in row
            for row in f
        )
```

5. Agent Chaining and Dependency Management

The Problem

Your morning workflow has a natural dependency: Process Inbox (8am) should ideally run *before* Daily Driver morning (6am) — or more likely, you'd want to adjust times so Process Inbox runs first. But `launchd` has no native concept of job dependencies for user agents.

Pattern 1: Schedule Staggering (Simplest)

The most pragmatic approach is to stagger schedules with enough buffer time:

Agent	Time	Buffer
Process Inbox	5:30 AM	—
Daily Driver (morning)	6:15 AM	45 min after inbox
Health Audit	7:00 PM	—
Daily Driver (evening)	7:30 PM	30 min after health

Set `max_turns` conservatively so agents complete within their buffer window. Your current default of 30 turns with a \$0.50 budget cap should keep runs under 10 minutes.[1]

Pattern 2: WatchPaths (Event-Driven)

`launchd` supports `WatchPaths` — a job can be configured to start when a specific file or directory changes. You can use this for dependency chaining:[20][21]

```
<!-- com.sean.agent.daily-morning.plist -->
<key>WatchPaths</key>
<array>
  <string>/Users/seanwinslow/Code-Brain/claude-code-superuser-pack/agen
</array>
```

The Process Inbox agent touches a signal file when it completes:

```
# At the end of process_inbox.py
signal_path = config.repo_root / "agents-sdk/signals/inbox-complete"
signal_path.touch()
```

The Daily Driver morning plist watches that path and fires when it changes. This is the cleanest native macOS solution, though note that

WatchPaths doesn't tell the launched job *which* path triggered it — your agent should check all watched paths.[20]

Pattern 3: Python Orchestrator (Most Flexible)

For complex dependency graphs, create a meta-agent that orchestrates other agents:

```
# agents/orchestrator.py
async def morning_workflow(config):
    """Run morning agents in dependency order."""
    # Phase 1: Data gathering (parallel)
    await asyncio.gather(
        run_agent("process_inbox", config),
        run_agent("health_audit", config),
    )

    # Phase 2: Synthesis (depends on phase 1)
    await run_agent("daily_driver", config, mode="morning")
```

This gives you full control over parallelism, error handling, and conditional execution (e.g., skip daily driver if inbox processing failed). The launchd plist triggers the orchestrator, which handles the rest.

Pattern 4: Completion Signals via Vault

A vault-native approach: each agent writes a completion marker to a known location (e.g., a YAML field in the daily note's frontmatter):

```
---
agents:
  inbox_processed: true
  inbox_processed_at: "2026-02-22T05:45:00"
  health_audit: false
---
```

Downstream agents read the frontmatter via `vault_io.read_frontmatter()` and decide whether to proceed or defer. [1]

6. Obsidian + AI Agent Integrations Beyond Note Creation

Several projects have pushed well beyond simple note creation into knowledge graph traversal, semantic search, and vault-wide pattern detection.

Smart Connections Plugin

The most mature Obsidian AI integration. Smart Connections v4 creates a local embedding index of your entire vault, enabling: [22] [23]

- **Semantic similarity scoring** — opens a sidebar showing notes related to your current note by meaning, not just keyword or link
- **RAG-powered chat** — when you query the integrated chat, it retrieves relevant vault notes via the embedding index and sends them as context to the LLM
- **Folder exclusion** — you can exclude folders (e.g., AI-generated content) from the index to keep it "your" knowledge

For your agent use case, Smart Connections' embedding model could serve as a **local semantic search layer** that agents query during runs. For example, the Weekly Review agent could search for notes semantically related to "blockers" or "decisions" rather than relying on exact tag matches.

Sonar Plugin (Local-First Agentic Chat)

Sonar is a newer plugin offering offline semantic search and agentic AI chat, powered by llama.cpp. Key capabilities: [24]

- **Hybrid retrieval** combining vector embeddings with BM25 for better coverage than vector-only search
- **Cross-encoder reranking** for higher precision on search results

- **Agentic chat** where the assistant can iteratively search, read, and edit notes via tool use
- **Custom tool extensions** via JavaScript modules (e.g., SearXNG web search, Google Calendar integration)
- **Automatic indexing** of Markdown, PDFs, and audio (via local Whisper transcription)

The agentic architecture is directly relevant to your setup — Sonar's model of "agent uses tools to search and edit vault" mirrors what your SDK agents do via `vault_inject`.

AI Knowledge Graph Approaches

A YouTube tutorial by an AI engineering educator demonstrates an ID-first workflow for turning an Obsidian vault into a queryable knowledge graph:[25]

- **Schema-driven notes** — hubs, concepts, and technology nodes with structured frontmatter
- **Transcript-to-graph pipeline** — system prompts that auto-create and link nodes from transcripts
- **Agent-traversable structure** — the graph's explicit IDs make every note instantly queryable by an AI agent that can check node contents and follow links

This approach would let your agents do things like: "Find all project notes linked to the 'blocked' concept" or "Traverse from the 16BitFit hub to find all decisions made this week."

Obsidian MCP Server

Your `agents-sdk` doc already lists the Obsidian MCP server as a medium-priority integration. This provides direct vault interaction via MCP — search, tag queries, and graph traversal — without going through raw file I/O. For agents that need to traverse the vault (e.g., Weekly Review aggregating across projects), this is more powerful than globbing markdown files.[1]

Local Semantic Indexing Proposals

The Obsidian community has an active proposal for a local semantic "Knowledge Indexer" as a core layer. The vision:[26]

- A shared embedding index that all plugins can query (eliminating the current fragmentation where each AI plugin maintains its own cache)
- Exposed via a local MCP service so agent frameworks can reason over the vault without exporting notes
- Capabilities like suggested backlinks, concept clustering, and knowledge synthesis

This isn't shipped yet, but it signals where the ecosystem is heading — and your agent architecture is well-positioned to adopt it when it arrives.

Practical Integration for Your Agents

The most actionable path for adding semantic search to your agent runs:

1. **Install Smart Connections** in your Obsidian vault to build the embedding index.
2. **Create a custom MCP tool** in `lib/custom_tools.py` that queries the Smart Connections index (it stores embeddings as JSON files in `.smart-connections/`).
3. **Add the tool to agents that benefit from semantic search** — Weekly Review (find thematic patterns across notes), Process Inbox (classify notes by semantic similarity to existing vault areas), and Health Audit (surface correlations between workout and nutrition notes).

Alternatively, if you prefer keeping everything in your Python stack, embed your vault using a local model (e.g., sentence-transformers via Hugging Face) and store vectors in a lightweight DB like ChromaDB. Your agents can then query this during runs without depending on Obsidian being open.

Implementation Priority Matrix

Based on your current architecture and the phases outlined in your agents-sdk doc, here's a suggested implementation order:

Priority	Enhancement	Effort	Impact
1	Add filelock to vault_io.inject_at_anchor()	30 min	Prevents all write conflicts
2	Build Preserve Session stop hook + queue system	2-3 hours	Captures every interactive session
3	CSV pre-processor for Spending Analysis	1-2 hours	Privacy-safe financial analysis
4	Add retry logic to base agent class	1 hour	API resilience for all agents
5	Stagger schedules + add WatchPaths for inbox → morning chain	1 hour	Dependency ordering
6	Smart Connections MCP tool for semantic search	3-4 hours	Vault-wide pattern detection

The first four items are foundational infrastructure that every agent benefits from. Items 5-6 are enhancements for specific workflows.

References

1. [agents-sdk.md](#) - # Agents SDK Integration Guide

How the Claude Agent SDK extends the Superuser Pack from an intera...

2. [Locking a file in Python - Stack Overflow](#) - I need to lock a file for writing in Python. It will be accessed from multiple Python

processes at o...

3. [filelock](#) - A platform-independent file locking library for Python, providing inter-process synchronization: fro...
4. [File locking in Python - tilde.town](#) - A lock is a way to tell other threads or processes that a resource (like a file) is in use. Unfortun...
5. [Everything you never wanted to know about file locking](#) - fcntl() locks are much more powerful than flock() locks. Like flock(), there are both shared and exc...
6. [Running Jujutsu with Claude Code Hooks - Matthew Sanabria](#) - Each hook event passes JSON data to your hook that can either be parsed and used or ignored entirely...
7. [Hooks reference - Claude Code Docs](#) - Claude Code captures a snapshot of hooks at startup and uses it throughout the session. ... Exit cod...
8. [Claude Code Hook Control Flow | Developing with AI Tools](#) - Hooks receive JSON data via standard input (stdin) that provides session and event-specific informat...
9. [Claude Code Hooks: A Practical Guide to Workflow Automation](#) - "Create a Python script that reads the hook input JSON, extracts the file content, and uses regex pa...
10. [Automate Your AI Workflows with Claude Code Hooks | Butler's Log](#) - Since we want to know when something is completed, we'll use the Stop hook, which fires when everyth...
11. [How I Built a Skill That Lets Me Talk to Claude's Conversation Memory](#) - How Claude Code Stores Conversations#. Every Claude Code session gets saved as a JSONL file in ~/.cl...
12. [Hooks architecture - Claude-Mem](#) - As of Claude Code 2.1.0 (ultrathink update), SessionStart hooks no longer display user-visible messa...
13. [Claude Code Plugin - memsearch - Zilliz](#) - The memsearch transcript command handles all the parsing, truncation, and formatting. The JSONL stru...
14. [clear command causes transcript file issues breaking Stop hooks ...](#) - When using the /clear command in Claude Code, the Stop hook fails to extract context from the conver...
15. [LLM Security: Top 10 Risks and 5 Best Practices - Tigera](#) - Data privacy measures, such as data anonymization and encryption, prevent the exposure of sensitive ...

16. [Safeguarding Data Privacy While Using LLMs | Tonic.ai](#) - Protect sensitive data when using LLMs with anonymization, secure handling, and GDPR/HIPAA complianc...
17. [LLM Data Privacy: Protecting Enterprise Data in the World of AI - Lasso](#) - In enterprise contexts, LLM data privacy focuses on protecting sensitive information that passes thr...
18. [launchd.plist\(5\) man page](#) - Multiple dictionaries may be specified in an array to schedule multiple calendar intervals. ... fail...
19. [how to setup sleep and wake schedule on your Mac using Terminal ...](#) - ... sleep or wake on a particular time and date. Apple removed the feature from system settings from...
20. [\[launchd-dev\] WatchPaths](#) - 1. user touches path A · 2. launchd launches the agent with path A, but it takes a while to start up...
21. [Creating Launch Daemons and Agents](#) - Explains how to write background processes that perform work on behalf of applications or serve cont...
22. [How To Add AI To Obsidian \(Free Private Local\) Smart Connections](#) - ... Smart Connections Plugin. New ideas, surfacing insights, making connections, distilling complex ...
23. [Just wanted to mention that the smart connections plugin is incredible.](#) - A sort of text classification algorithm called a 'text embedding model' scans your vault and can aut...
24. [\[ANN\] Sonar: Offline semantic search and agentic AI chat for ...](#) - Sonar is a local-first semantic search + agentic chat plugin for Obsidian, powered by llama.cpp. Eve...
25. [Learn Faster with an AI Knowledge Graph in Obsidian - YouTube](#) - With this system you'll learn how to organize knowledge, structure your vault, and make AI agents re...
26. [Local Semantic “Knowledge Indexer” \(Opt-In Core Layer\) + Optional ...](#) - Obsidian currently excels at representing explicit links, but cannot illicit semantic relationships ...